Streamlining symbol files in Oberon

Andreas Pirklbauer

1.11.2026

1. Overview

This technical note presents a simplification and improvement of the handling of import and export[1] for the *Project Oberon 2013* system, which is a reimplementation of the original *Oberon* operating system on an FPGA development board in 2013, as published at *www.projectoberon.com.* If you use the *Extended Oberon*[2] system, the improvements presented in this document are already implemented by default.

2. Brief historical context

The topic of *symbol files* (=module interface files) has accompanied compiler development ever since the original *module* concept with *separate compilation* and type-checking *across* module boundaries (as opposed to *independent* compilation, where no such checks are performed) was introduced in the 1970s and adopted in languages such as Mesa, Ada, Modula-2 and Oberon.

A correct implementation of the *module* concept was by no means obvious initially. However, the concept has evolved and today, simple implementations exist covering all key requirements, e.g.,

1. *Re-export conditions*: Imported types may be *re-exported* and their *imports* may be hidden.
2. *Recursive data structures*: Pointer declarations may *forward reference* a record type.
3. *Cyclic references:* Record and pointer types may contain cyclic references among themselves.
4. *Module alias names*: A module can be imported under a different (alias) name.
5. *Hidden record fields:* Offsets of non-exported pointer fields are needed for garbage collection; offsets of non-exported procedure variable fields are needed for module reference checking.

A careful and detailed study of the evolution that led to today's status quo – which contains many useful lessons and is therefore well worth the effort – is far beyond the scope of this technical note. The reader is referred to the literature. Here, a very rough sketch must suffice:

- Module concept introduced in 1972, early languages include Mesa, Modula-2 and Ada [1].
- Modula-2 implementation on PDP-11 in 1979 already used *separate* compilation [2].
- Modula-2 implementation on Lilith in 1980 already used *separate* compilation [3].
- The first *single-pass* Modula-2 compiler in 1984 used a *postorder* traversal of the symbol table [4, 5, 7].
- Oberon compilers in the 1990s used either a *postorder* or *preorder* traversal of the symbol table [8-12].
- The Oberon on ARM compiler in 2008 used a *fixup* technique for types in symbol files [13].
- The Project Oberon 2013 compiler uses *preorder* traversal and a *fixup* technique for types [14].

As with the underlying languages, all these re-implementations and refinements of the handling of import and export (and the associated symbol files) are characterized by a continuous reduction of complexity.

In this note, we present yet another simplification by eliminating the so-called "fixup" technique for *types* during export and subsequent import, as well as some additional improvements.

3. Code that can be omitted in Project Oberon 2013

The Oberon system and compiler were re-implemented around 2013 on an FPGA development board. The compiler was derived from an earlier version of the Oberon compiler for the ARM processor. In the Project Oberon 2013 compiler, the same "fixup" technique to implement forward references in symbol files as in the ARM Oberon compiler is used. The following passage from the *Oberon on ARM* report [13] describes this technique:

[1] *http://www.github.com/andreaspirklbauer/Oberon-module-imports*
[2] *http://www.github.com/andreaspirklbauer/Oberon-extended*

If a type is imported again and then discarded, it is mandatory that this occurs before a reference to it is established elsewhere. This implies that types must always be defined before they are referenced. Fortunately, this requirement is fulfilled by the language and in particular by the one-pass strategy of the compiler. However, there is one exception, namely the possibility of forward referencing a record type in a pointer declaration, allowing for recursive data structures:

```
TYPE P = POINTER TO R;
  R = RECORD x, y: P END
```

Hence, this case must be treated in an exceptional way, i.e. the definition of P must not cause the inclusion of the definition of R, but rather cause a forward reference in the symbol file. Such references must be fixed up when the pertinent record declaration had been read. This is the reason for the term {fix} in the syntax of (record) types. Furthermore, the recursive definition

```
TYPE P = POINTER TO RECORD x, y: P END
```

suggests that the test for re-import must occur before the type is established, i.e. that the type's name must precede the type's description in the symbol file, where the arrow marks the fixup:

```
TYP [#14 P form = PTR [^1]]
TYP [#15 R form = REC [^9] lev = 0 size = 8 {y [^14] off = 4 x [^14] off = 0}] → 14
```

The excerpt above notes that types must always be defined before they are referenced during compilation. At first sight, this might suggest the need for fixups to handle forward or cyclic references.

Recall, however, that during compilation the compiler's symbol table is built while processing declarations, whereas symbol files are written at the end of the compilation process, when all forward references have been resolved in the symbol table. An alternative approach, therefore – described in [9] and [12] and in fact already implemented in Project Oberon 2013 – structures the symbol file such that each unique type reference always precedes its corresponding type description. This arrangement is possible even in the presence of forward or cyclic dependencies.

The principle is that the linear description of an enclosing object *P* may open a "parenthesis" enclosing the complete description of a component object *R* (such as a record referenced by a pointer type, a record field within a record, or a parameter of a procedure type). After the parenthesis is closed, the description of *P* resumes. Importantly, if the description of *R* in turn refers back to *P*, the reference number assigned to *P* is used, thereby resolving the cyclic dependency without fixups.

The example above of a forward pointer declaration is now linearized as follows, without any fixups:

```
TYP [#14 P form = PTR [#15 R form = REC [^9] lev = 0 size = 8 {y [^14] off = 4 x [^14] off = 0}]]
```

Here, the pointer declaration *P* obtains a reference number #14 which precedes its actual type description, while the record declaration *R* with reference number #15 is *embedded* in the linear description of *P*. This eliminates the need for fixups. *R* refers back to the description of *P* using reference number #14.

As a result of this approach, all type references in symbol files inherently take the form of *backward* references. This design choice also simplifies the process of reconstructing the symbol table data structure during import, and makes it straightforward to ensure that types are always *defined* (i.e. inserted into the symbol table) before they are *referenced* (see *ORB.OutType* and *ORB.InType*):

```
PROCEDURE OutType(VAR R: Files.Rider; t: Type);
  ...
BEGIN
  IF t.ref > 0 THEN (*type was already output*) Write(R, -t.ref)
  ELSE obj := t.typobj;
    IF obj # NIL THEN Write(R, Ref); t.ref := Ref; INC(Ref) ELSE Write(R, 0) END ;
    Write(R, t.form);
    IF t.form = Pointer THEN OutType(R, t.base)
    ELSIF t.form = Array THEN OutType(R, t.base); ...
    ELSIF t.form = Record THEN OutType(R, t.base); ...
    ELSIF t.form = Proc THEN OutType(R, t.base); ...
```

```
    END
  END ;
  ...
END OutType;

PROCEDURE InType(VAR R: Files.Rider; thismod: Object; VAR T: Type);
  ...
BEGIN Read(R, ref);
  IF ref < 0 THEN T := typtab[-ref]  (*type was already read*)
  ELSE NEW(t); T := t; typtab[ref] := t; ...
    Read(R, form); t.form := form;
    IF form = Pointer THEN InType(R, thismod, t.base); ...
    ELSIF form = Array THEN InType(R, thismod, t.base); ...
    ELSIF form = Record THEN InType(R, thismod, t.base); ...
    ELSIF form = Proc THEN InType(R, thismod, t.base); ...
    END
  END ;
  ...
END InType;
```

One can easily verify that in Project Oberon 2013 types are *always* already "fixed up" with the right value by slightly modifying the procedure *ORP.Import* to print an error message when types are *not* fixed up, i.e.

```
WHILE k # 0 DO
  IF typtab[k].base # t THEN ORS.Mark("type not fixed up") END ;
  typtab[k].base := t; Read(R, k)
END
```

The error message will *never* be printed when importing a module. A more formal proof can of course easily be constructed. It rests on the observation that a type's *reference* number is written to the symbol file *before* the remaining type description and also before any other types or objects refer to this type.

As a result of these observations, the following code in red in *ORB.Import* and *ORB.Export* can be omitted:

```
PROCEDURE Import*(VAR modid, modid1: ORS.Ident);
  ...
BEGIN
  ...
  Read(R, class);
  WHILE class # 0 DO
    NEW(obj); obj.class := class; Files.ReadString(R, obj.name);
    InType(R, thismod, obj.type); obj.lev := -thismod.lev;
    IF class = Typ THEN
      t := obj.type; t.typobj := obj; Read(R, k);
      (*fixup bases of previously declared pointer types*)
      WHILE k # 0 DO typtab[k].base := t; Read(R, k) END
    ELSE ...
      IF class = Const THEN ...
      ELSIF class = Var THEN ...
      END
    END
    obj.next := thismod.dsc; thismod.dsc := obj; Read(R, class)
  END ;
  ...
END Import;

PROCEDURE Export*(VAR modid: ORS.Ident; VAR newSF: BOOLEAN; VAR key: LONGINT);
  ...
BEGIN
  ...
  obj := topScope.next;
  WHILE obj # NIL DO
    IF obj.expo THEN Write(R, obj.class); Files.WriteString(R, obj.name);
      OutType(R, obj.type);
      IF obj.class = Typ THEN
        IF obj.type.form = Record THEN obj0 := topScope.next;
          (*check whether this is base of previously declared pointer types*)
```

```
          WHILE obj0 # obj DO
            IF (obj0.type.form = Pointer) & (obj0.type.base = obj.type)
              & (obj0.type.ref > 0) THEN Write(R, obj0.type.ref) END ;
            obj0 := obj0.next
          END
        END ;
        Write(R, 0)
      ELSIF obj.class = Const THEN ...
      ELSIF obj.class = Var THEN ...
      END
    END ;
    obj := obj.next
  END ;
  ...
END Export;
```

4. Handling type alias names among imported and re-imported modules correctly

In Project Oberon 2013, compilation of module *C* below leads to a compilation error.

```
MODULE M;
  TYPE A* = RECORD END ;
    B* = A;
END M.

MODULE M0;
  IMPORT M;              (*import type M.A and type alias name M.B*)
  VAR a*: M.A;           (*re-export type M.A, but the type name M.B is (incorrectly) written in PO 2013*)
END M0.

MODULE C;
  IMPORT M0, M;          (*first re-import type M.A via M0 and then directly import type M.B from M*)
  VAR c: M.A;            (*compilation error in Project Oberon 2013 if explicit import of M were allowed*)
END C.
```

The first reason is that the explicit import of module *M* in module *C* is not allowed in Project Oberon 2013, as module *C* has already re-imported the type *M.A* via *M0*, resulting in an *invalid import order* error.

But even if it were permitted, this scenario would trigger a compilation error during compilation of module *C* due to the unknown type *M.A* when processing the declaration of the global variable *c*. The issue arises from the import process in module *M0*, where the imported type alias *M.B* is correctly identified as the previously re-imported type *M.A*. However, the subsequent assignment *t.typobj := obj* in *ORB.Import* incorrectly redirects the back-pointer for *M.B* to the newly imported type alias object *M.B*, instead of leaving it pointing to *M.A*. Consequently, when *M.A* is re-exported by module *M0*, it is misrepresented as *M.B* in the symbol file of *M0*, rendering the original type *M.A* inaccessible to clients like module *C*.

To solve this issue, it suffices to replace the following statement in *ORB.Import*:

```
t.typobj := obj
```

with:

```
IF t.typobj = NIL THEN t.typobj := obj END
```

which establishes the *typobj* back-pointer only if it doesn't exist yet. This ensures that an imported type alias name continues to point to the *original* imported type object, not another imported type alias object.

This is the same type of precaution as in *ORP.Declarations*, where type aliases declared in the module being compiled are initialized as follows:

```
IF tp.typobj = NIL THEN tp.typobj := obj END
```

which has the same effect for type alias names declared in the module *currently* being compiled.

As this example shows, special care must be taken when handling cases in which *imported* or *re-imported* types have type alias names associated with them. Consider the following scenario:

```
MODULE M;
  TYPE A* = RECORD END ;        (*exported original type*)
    B = A;                      (*non-exported type alias name of an exported original type A*)
    C* = B;                     (*exported type alias name of (another alias of) an exported original type A*)
    D = RECORD END ;            (*non-exported original type, considered local to module M*)
    (*E* = D;*)                 (*export of E not allowed, because the original type D is not exported*)
END M.

MODULE M0;
  IMPORT M;
  TYPE F* = M.C;                (*type alias names of an imported type alias name M.C, resolve to M.A*)
    G = M.C;                    (*type alias name of an imported type alias name M.C, resolves to M.A*)
  VAR c*: M.C;                  (*original type M.A re-exported, not the type alias name M.C*)
    f*: F;                      (*original type M.A re-exported, as well as the type alias name F*)
    g*: G;                      (*original type M.A re-exported, but not the type alias name G*)
END M0.

MODULE C;
  IMPORT M0, M;                 (*original type M.A re-imported via M0 and explicitly imported from M*)
  VAR c*: M.C;                  (*type alias name M.C of the original type M.A directly imported from M*)
END C.

MODULE D;
  IMPORT M0;                    (*original type M.A encountered only via a type alias name in M0*)
  VAR f*: M0.F;                 (*type alias name M0.F of the original type M.A directly imported from M0*)
END D.
```

In our implementation, type alias names are handled as follows:

- A *type alias name* is simply an additional name for an existing type; it does not define a new type. A type may have multiple alias names, and if a type alias name refers to another type alias name, it ultimately resolves to the original type.

- A type alias name can only be exported if the *original type* is also exported or was itself imported (in which case it was originally exported). Allowing the export of a type alias name of a non-exported type would effectively expose a type not intended to leave the module and could alter semantics[3].

- If a type alias name of an imported type is exported, the compiler always re-exports the original type as well. For example, exporting the type alias name *F* in module *M0* causes the original type *M.A* to be re-exported by *M0*, in addition to the type alias name *F*. This ensures that all references to *M.A*, whether accessed directly through *M* or indirectly via *M0*, refer to the same type object in the compiler's symbol table, thereby preserving type identity across modules.

- If a type alias name of an imported type is not exported but is used in an exported variable of that type, the compiler only re-exports the original type. For example, exporting the variable *g* in module *M0* causes only the original type *M.A* to be re-exported by *M0*, but not the type alias name *G*.

- If a client module imports a type alias name and then exports or re-exports a variable of that type, the re-export is again expressed in terms of the original type, not the type alias name. For example, module *C* directly imports the type alias name *M.C* from *M*. When *C* exports the global variable *c* of that type, the original type *M.A* is re-exported by *C*, not the type alias name *M.C*.

- If a client module accesses a type *only* through a type alias name imported from an *intermediate* module and then exports a variable of that type, the re-export is again expressed in terms of the original type, not the intermediate type alias name. For example, module *D* imports the type alias name *M.F* from *M0* (and with it the original type *M.A* from *M*). When *D* exports the global variable *f* of that type, the original type *M.A* is re-exported by *D*, not the type alias name *M.F*.

[3] *As an alternative design, the export of type alias names could be disallowed entirely, restricting them to purely local use, similar to module alias names.*

The rules for exporting type alias names form part of the broader export rules. For completeness, we note the following *general export rules*, as realized in our implementation:

- A type alias name can only be exported if the *original type* is also exported or was itself imported.
- An array type or a variable of an array type can only be exported if the array *base* type is exported.
- A record field can only be exported if the type of the *field* and the type of the *record* itself are exported.
- A procedure can only be exported if the types of *formal parameters* and the *return type* are exported.
- A pointer type or a pointer variable *can* be exported *even if* its pointer base type is not exported.
- An extended record type *can* be exported *even if* its record base type is not exported.
- String constants are exported as read-only (Extended Oberon only).

5. Allowing re-imports to co-exist with module alias names and globally declared identifiers

In Project Oberon 2013, compilation of modules *M1* and *M2* below leads to a name conflict with the re-imported module name *M*:

```
MODULE M;
  TYPE T* = RECORD END ;
END M.

MODULE M0;
  IMPORT M;
  VAR t*: M.T;                    (*re-export type M.T*)
END M0.

MODULE M1;
  IMPORT M0;                      (*re-import type M.T*)
  VAR M: INTEGER;                 (*name conflict with globally declared identifier in Project Oberon 2013*)
END M1.

MODULE M2;
  IMPORT M := M0;                 (*name conflict with module alias name in Project Oberon 2013*)
END M2.
```

To solve this issue, we *hide* re-imported modules from the global namespace, allowing them to coexist with global identifiers and module alias names of explicitly imported modules. This is achieved by simply *skipping* over re-imports (identified as *~obj.rdo*) in various loops in module *ORB:*

```
PROCEDURE NewObj*(VAR obj: Object; id: ORS.Ident; class: INTEGER);
  VAR new, x: Object;
BEGIN x := topScope;
  WHILE (x.next # NIL) & ((x.next.name # id) OR (x.next.class = Mod) & ~x.next.rdo) DO
    x := x.next
  END ;
  ...

PROCEDURE thisObj*(): Object;
  VAR s, x: Object;
BEGIN s := topScope;
  REPEAT x := s.next;
    WHILE (x # NIL) & ((x.name # ORS.id) OR (x.class = Mod) & ~x.rdo) DO
      x := x.next
    END ;
    ...

PROCEDURE ThisModule(name, orgname: ORS.Ident; decl: BOOLEAN; key: LONGINT): Object;
  VAR mod: Module; obj, obj1: Object;
BEGIN obj1 := topScope;
  IF decl THEN obj := obj1.next;  (*search for alias, obj.class = Mod implicit*)
    WHILE (obj # NIL) & ((obj.name # name) OR ~obj.rdo) DO obj := obj.next END
    ...
```

If a re-imported module is subsequently explicitly imported, its entry in the symbol table is converted to an explicit import. This ensures that its name cannot co-exist with module alias names or global identifiers.

6. Allowing reusing the original module name if a module has been imported under an alias name

The Oberon language report defines aliased module imports as follows: *If the form "M := M1" is used in the import list, an exported object x declared within M1 is referenced in the importing module as M.x.*

Unfortunately, this definition allows for several different interpretations, for example:

Interpretation A:

- *It is module M1 that is imported, not M*
- *The module alias name M renames module M1 and the original name M1 can be reused*
- *A module can only have a single module alias name*

Interpretation B:

- *It is module M1 that is imported, not M*
- *A module alias name M is just an additional name for M1 and the original name M1 cannot be reused*
- *A module can have multiple module alias names*

Interpretation C:

- *It is module M that is imported, but the file with name M1 is read*

In our implementation, we have adopted interpretation A. It implies that the statement *IMPORT M := M1* imports module *M1* and associates the local name *M* with it, i.e. the identifier *M* is used as usual, but the file with name *M1* is read. The alias name *M* is substituted for the original name *M1* (which can be reused).

For example, the following scenarios are all legal:

```
MODULE A1; IMPORT M0 := M1, M1 := M2; END A1.
MODULE A2; IMPORT M0 := M1, M1 := M0; END A2.
MODULE A3; IMPORT M0 := M1, M2 := M0; END A3.
```

whereas the following scenario is illegal:

```
MODULE B1; IMPORT M1, A := M1, B := M1; END B1.
```

This is implemented by *not* checking the two combinations *obj.orgname # name* and *obj.name # orgname*, where *obj* denotes an existing module in the module list of the symbol table. Not checking the combination *obj.orgname = name* allows the second import *M1 := M2* with *name = M1* in module *A1*. Not checking the combination *obj.name = orgname* allows the second import *M1 := M0* with *orgname = M0* in module *A2*. This leaves us with checking the *other* two combinations *obj.name # name* and *obj.orgname # orgname*:

```
PROCEDURE ThisModule(name, orgname: ORS.Ident; decl: BOOLEAN; key: LONGINT): Object;
  VAR mod: Module; obj, obj1: Object;
BEGIN obj1 := topScope;
  IF decl THEN obj := obj1.next;  (*search for alias*)
    WHILE (obj # NIL) & ((obj.name # name) OR ~obj.rdo) DO obj := obj.next END
  ELSE obj := NIL
  END ;
  IF obj = NIL THEN obj1 := obj1.next;  (*search for module*)
    WHILE (obj # NIL) & (obj.orgname # orgname) DO obj1 := obj; obj := obj1.next END;
    IF obj = NIL THEN (*insert new module*) ...
    ELSE (*module already present*)
      IF decl THEN (*explicit import by declaration*)
        IF obj.rdo THEN ORS.Mark("mult def")
        ELSE obj.name := name; obj.rdo := TRUE  (*convert to explicit import*)
        END
      END
    END
  ELSE ORS.Mark("mult def")
  END ;
  RETURN obj
END ThisModule;
```

7. Propagating imported export numbers of type descriptor addresses to client modules

The Project Oberon 2013 implementation does not permit type tests or type guards on types that are only re-imported through other modules, but are not explicitly imported. This doesn't pose an issue since only explicitly imported types can be referenced by name in client modules anyway.

But our implementation permits an explicit import of a module *M* even if types from *M* have already been re-imported through other modules, as described in the next section. In such cases, the previously re-imported module's entry in the compiler's symbol table is simply converted to an explicit import. To enable type tests and type guards on types declared in such "converted" modules, we have replaced the following code in *ORB.OutType*:

```
ELSIF t.form = Record THEN ...
  IF obj # NIL THEN Files.WriteNum(R, obj.exno) ELSE Write(R, 0) END ;
  ...
```

with:

```
ELSIF t.form = Record THEN ...
  IF obj # NIL THEN
    IF t.mno > 0 THEN Files.WriteNum(R, t.len) ELSE Files.WriteNum(R, obj.exno) END
  ELSE Write(R, 0)
  END ;
```

This makes sure that *imported* export numbers of type descriptor addresses (stored in the field *t.len*) are re-exported to client modules, thereby enabling type tests and type guards on such types (in case the declaring module is also *explicitly* imported).

8. Allowing an explicit import after previous re-imports of types of the same module

In the Oberon programming language, imported types can be re-exported, and their original import may be hidden from the re-importing module during the *import process*. This means that a type *T* from one module *(M)* can be imported by another module *(M1)* and then re-exported to a third module *(M2)* without *M2* being aware of the original import from *M*. Two common approaches to implementing this mechanism include:

1. *Self-contained symbol files*: In this approach, imported types are included in symbol files, making them self-contained. This eliminates recursive imports, but results in (somewhat) larger symbol files.

2. *Recursive imports*: In this approach, all required symbol files are imported recursively in full. This results in smaller symbol files, but requires more imports and makes module dependencies explicit.

Project Oberon 2013 has opted for the first approach (self-contained symbol files). However, it does not allow an explicit import of a module *M* after types of *M* have previously been re-imported via other modules. Our implementation removes this limitation. Consider the following scenario*:*

```
MODULE M;                          (*base module*)
  TYPE T0* = RECORD END ;
    T1* = RECORD END ;
    T2* = RECORD END ;
END M.

MODULE M0;                         (*intermediate module*)
  IMPORT M;                        (*import types M.T0, M.T1 and M.T2 from M directly*)
  VAR t0*: M.T0;                   (*re-export type M.T0 to clients of M0*)
END M0.

MODULE M1;                         (*intermediate module*)
  IMPORT M;                        (*import types M.T0, M.T1 and M.T2 from M directly*)
  VAR t0*: M.T0;                   (*re-export type M.T0 to clients of M1*)
    t1*: M.T1;                     (*re-export type M.T1 to clients of M1*)
END M1.
```

where the modules *M*, *M0* and *M1* are imported by the client modules *A*, *B* and *C*, respectively, as follows:

```
MODULE A;                          (*allowed in Project Oberon 2013 and in Extended Oberon*)
  IMPORT M0,                       (*re-import type M.T0 via M0*)
    M1;                            (*re-import types M.T0 and M.T1 via M1*)
END A.

MODULE B;                          (*allowed in Project Oberon 2013 and in Extended Oberon*)
  IMPORT M,                        (*import types M.T0, M.T1 and M.T2 from M directly*)
    M0,                            (*re-import type M.T0 via M0*)
    M1;                            (*re-import types M.T0 and M.T1 via M1*)
END B.

MODULE C;                          (*not allowed in Project Oberon 2013, but allowed in Extended Oberon*)
  IMPORT M0,                       (*re-import type M.T0 via M0*)
    M1,                            (*re-import types M.T0 and M.T1 via M1*)
    M;                             (*import types M.T0, M.T1 and M.T2 from M directly*)
END C.
```

A robust implementation must correctly handle imported types in all possible scenarios, including explicit imports and re-imports from multiple symbol files, in any combination and order. The principal requirement is to ensure that, regardless of how many times a particular type is encountered during the import process and irrespective of the order of the various (re-)imports of this type, it consistently maps to a *single* node in the symbol table of the compiler. Failure to achieve this can lead to incompatibilities during *type checking*. Recall that in typical implementations, type equality is determined by comparing pointers referencing type descriptors in the symbol table. This is made possible by the Oberon language definition, which specifies type equivalence based on names rather than structure.

In the above example, the type *M.T0* is *re-imported* twice during compilation of module *A* – first, when module *M0* is imported, and second, when module *M1* is imported – while the symbol file of the declaring module *M* is never loaded. During compilation of module *B*, the type *M.T0* is first *explicitly* imported when the symbol file of module *M* is loaded and then *re-imported* twice – first, when module *M0* is imported and second, when module *M1* is imported. During compilation of module *C*, the type *M.T0* is first *re-imported* twice via modules *M0* and *M1* and subsequently *explicitly* imported from module *M*.

The Project Oberon 2013 implementation already has a built-in mechanism to identify instances where a type being re-imported is already present in the compiler's symbol table. This may be the case because the symbol file of the module defining the type has already been loaded, or because the type has already been read when loading other symbol files.

The implementation is straightforward and involves a simple search for the type's name within the object list of the declaring module *M* (see *ORB.InType*):

```
IF modname[0] # 0X THEN (*re-import*)
  Files.ReadInt(R, key); Files.ReadString(R, name);
  mod := ThisModule(modname, modname, FALSE, key);
  obj := mod.dsc;  (*search type*)
  WHILE (obj # NIL) & (obj.name # name) DO obj := obj.next END ;
  IF obj # NIL THEN T := obj.type  (*type object found in object list of mod*)
  ELSE (*insert new type object in object list of mod*)
    NEW(obj); obj.name := name; obj.class := Typ; obj.next := mod.dsc;
    mod.dsc := obj; obj.type := t; t.mno := mod.lev; t.typobj := obj; T := t
  END ;
  typtab[ref] := T
END
```

This mechanism covers the case where a type is *first explicitly imported and then re-imported* via other modules (once or several times), as well as the case where a type is re-imported solely via other modules (once or several times) without its declaring module ever being explicitly imported.

But it does not cover the case where a type is *first re-imported via other modules (once or several times) and then explicitly imported from its declaring module*, as in module *C* above. Instead, the Project Oberon

2013 implementation enforces a restriction: It prevents explicit imports of modules from which individual types have previously been re-imported via other modules.

If we want to allow *explicit* imports *after* prior re-imports of the same type, we could of course employ the same technique that is already used for handling *re-imports*.

```
IF modname[0] # 0X THEN (*re-import*)
  ... see above ...
ELSE (*explicit import*)
  Search the type in the object list of the currently imported module. If the
  type is found, map it to the type node of this (previously re-imported) type.
END
```

But this would involve searching for each explicitly imported type within the object list of the *currently* imported module *M*, just to find types that were previously re-imported via *other* modules. Furthermore, it would require additional modifications to ensure that the type's name can be accessed in *ORB.InType*.

An alternative approach consists of propagating the *original* reference number of a re-exported type *t*, denoted as *t.orgref*, across the module hierarchy and using this reference number to initialize the compiler's *type translation table*[4] before importing the type explicitly from the original module. This eliminates the need to search for each explicitly imported type within the compiler's symbol table.

The original reference number of a type *t* is propagated as follows:

- If a type *t* is originally exported by a module *M* and directly imported by an intermediate module *M0*, the field *t.orgref* is set to the reference number of "t in M" (read from the symbol file of module *M*). This effectively marks the beginning of the chain of re-exports and subsequent re-imports of this type[5].

- If *t* is re-exported by module *M0*, a new reference number for the type "t in M0" *(t.ref)* is assigned to it and written to the symbol file of *M0*, together with the original reference number of "t in M" (*t.orgref*).

- If *t* is subsequently *re-imported* by a client module *C* via the intermediate module *M0*, the field *t.orgref* is again set to the reference number of "t in M" (but this time read from the symbol file of module *M0*).

Note that the field *t.orgref* is written to symbol files only if *t* is a named type that is *re-exported*. Since re-exports are rare, this has a negligible effect on the overall length of symbol files.

With this preparation, our implementation approach can be summarized as follows:

- When a module *M* exports a type *t* to an intermediate module *M0* and a client module *C* subsequently re-imports this type via module *M0*, a module object for its declaring module *M* and a type object for the re-imported type in the object list of module *M* are inserted into the compiler's symbol table during compilation of *C*, together with its original reference number (*t.orgref*) in its declaring module *M*.

- If the same client *C* later also *explicitly* imports module *M*, we start by initializing the compiler's type translation table for module *M* with all types of *M* that have previously been re-imported via other modules, using the original reference numbers in their declaring module *M* as the index (this is why they are propagated). In the above example, these are the types *M.T0* and *M.T1*.

- For convenience, we also *mark* each previously re-imported type (e.g., by temporarily inverting the sign of its module number) during this initialization phase. This will allow us to easily detect whether a type read from the symbol file of *M* has previously been re-imported via *other* modules.

- If module *C* reads a type *t* from the symbol file of module *M* directly, there are two cases:

  Case A: The type *t* has *not* previously been re-imported via other modules (the frequent case). In this scenario, we create a new type object for *t* in the object list of module *M*. In the above example, this is the case when module *C* reads the type *M.T2* from the symbol file of *M*.

[4] *The compiler's type translation table (typtab) for a module M is a table containing references to all types of M that already exist in the object list of M.*
[5] *For the re-export mechanism to function, the symbol file of M must be read entirely at least once before initiating the chain of re-exports and re-imports of individual types of M.*

Case B: The type *t has* previously been re-imported via other modules. In this scenario, we reuse the existing type object found in the object list of the declaring module *M*, while continuing to read the type information for *t* from the symbol file of *M*. In the above example, this is the case when module *C* reads the types *M.T0* and *M.T1* from the symbol file of *M*. Since named types are written to symbol files *before* variables and procedures that refer to them, we know that the object *class* must be *Typ* in this case, and therefore no additional data needs to be read from the symbol file of *M*.

The following code excerpts show a possible implementation of this scheme:

ORB.Import:

```
thismod := ThisModule(modid, modid1, TRUE, key);
FOR i := Record+1 TO maxTypTab-1 DO typtab[i] := NIL END ;
obj := thismod.dsc;  (*initialize typtab with already re-imported types*)
WHILE obj # NIL DO
  typtab[obj.type.orgref] := obj.type;  (*initialize typtab*)
  obj.type.mno := -obj.type.mno;  (*mark type as re-imported*)
  obj := obj.next
END ;
...
Read(R, class);
WHILE class # 0 DO
  Files.ReadString(R, name); InType(R, thismod, t);
  IF t.mno < 0 THEN (*already re-imported*)
    t.mno := -t.mno; t.typobj.lev := -thismod.lev
  ELSE (*insert new object in object list of thismod*)
    NEW(obj); obj.class := class; obj.name := name;
    obj.type := t; obj.lev := -thismod.lev;
    IF class = Const THEN ...
    ELSIF class = Var THEN ...
    ELSIF t.typobj = NIL THEN t.typobj := obj
    END ;
    obj.next := thismod.dsc; thismod.dsc := obj
  END ;
  Read(R, class)
END
```

ORB.InType:

```
IF ref > 0 THEN
  Files.ReadString(R, modname);
  IF modname[0] # 0X THEN (*re-import*) ...
    Files.ReadInt(R, key); Files.ReadString(R, name); Read(R, orgref);
    mod := ThisModule(modname, modname, FALSE, key);
    obj := mod.dsc;  (*search type*)
    WHILE (obj # NIL) & (obj.name # name) DO obj := obj.next END ;
    IF obj # NIL THEN T := obj.type  (*type object found in object list of mod*)
    ELSE (*insert new type object in object list of mod*)
      NEW(obj); obj.name := name; obj.class := Typ; obj.next := mod.dsc;
      mod.dsc := obj; obj.type := t; t.mno := mod.lev; t.typobj := obj;
      t.typmod := mod; t.orgref := orgref
    END
  ELSE (*explicit import*)
    IF typtab[ref] # NIL THEN T := typtab[ref] END (*reuse already re-imported type*)
  END
END
```

The attentive reader may have wondered whether it is necessary to re-export the type name to the symbol file in *ORB.OutType*. Since the original reference number *t.orgref* already uniquely identifies a type, it could be used to search for the type in the object list of a re-imported module, and the type name could therefore be inserted into the symbol table only if the module is later explicitly imported in *ORB.Import*. However, an experiment has shown that the resulting reduction in the length of the symbol file is negligible, mainly because re-exports are (and should be) rare. Such a refinement is worthwhile only if the module anchor is written for *every* type, rather than only for re-exported ones, as described, for example, in [9]. We have therefore refrained from introducing this optimization and opted for the simpler solution of writing *both* the type name *and* its original reference number.

9. Recognizing multiple identifiers that appear on the same line as having the same type

In order to recognize multiple identifiers that appear on the same line in a record field list, a variable declaration or a formal parameter list as being of the same type, even if their (common) type is anonymous and subsequently exported and imported by other modules, we have decided to write a reference number to the symbol file for *each* pointer, array or record type encountered during the export process, rather than just for *named* types. This brings the compiler in line with Section 7 (variable declarations) of the language definition and allows, for example, the compilation of module *B* below.

```
MODULE A;
  TYPE R* = RECORD
    a*, b*: RECORD i*: INTEGER END  (*common type of fields a and b is anonymous*)
  END ;
END A.

MODULE B;
  IMPORT A;
  VAR x: A.R;
  PROCEDURE Go*;
  BEGIN x.a := x.b  (*<-- this assignment is now possible*)
  END Go;
END B.
```

A (small) downside of this approach is that slightly more reference numbers may be generated and written to the symbol file than *strictly* necessary, for example in cases where only one identifier appears on a given line, in which case a reference number is unnecessary. Note that we deliberately avoid writing a reference number for anonymous procedure types, since such types are compared by structure rather than by name (see *ORP.CompTypes*). Consequently, we also avoid writing a reference number for *declared* procedures, thereby helping to keep the symbol file small.

Our approach may, however, require additional entries in the compiler's type translation table (*ORB.typtab*) during subsequent imports. This overhead turns out to be negligible, typically amounting to just a few additional entries, and in many cases there are none at all. Accordingly, in our implementation, we have chosen the following (heuristic) criterion for writing a reference number to the symbol file, which has worked well in practice (see *ORB.OutType*):

```
(t.typobj # NIL) OR (t.form IN {Pointer, Array, Record})
```

An alternative would have been to determine *exactly* when a reference number is necessary, for example, by tracking *how many* identifiers appear on the same line in a record field list, a variable declaration, or a formal parameter list, and by writing a reference number only if more than one identifier does. For procedure types, reference numbers would be generated and written only for *procedure variables*, but not for *declared* procedures.

This alternative, however, would yield only marginal gains and would therefore not justify the additional implementation complexity. In fact, an experiment has shown that writing reference numbers only when necessary may even increase the size of the symbol file in some cases. This is because a reference count would then also have to be written to the symbol file for each record field list, variable declaration, or formal parameter list containing more than one identifier, in order to preserve this information and propagate it across the module hierarchy through the re-export mechanism

10. Writing the module anchor of re-exported types before the type description in the symbol file

When implementing the re-export mechanism through *self-contained symbol files*, it is essential to include in the type description a reference to the module in which a re-exported type was originally defined, i.e. the type's declaring module. This reference, known as the *module anchor*, typically includes the module name and key of the corresponding module. When combined with the type's name, it forms a unique *identifier* for the re-exported type. Note that in our implementation, this module anchor is written only for re-exported types. For types declared in (and exported by) the currently compiled module, a reference to the declaring module is written only once to the symbol file.

In contrast to Project Oberon 2013, our implementation writes this unique identifier of a re-exported type immediately *after* the type's reference number, but *before* the actual type description to the symbol file of the re-exporting module. This approach corresponds to postulate #5 in [5]. It guarantees that no other types can appear between the type's *reference* number and its *identification*, thereby ensuring that the code also works in the presence of cyclic references *among re-imported types.*

Recall that a type may refer to itself, as illustrated in the following example:

```
MODULE M;
  TYPE P1* = POINTER TO R1;
    P2* = POINTER TO R2;
    P3* = POINTER TO R3;
    R1* = RECORD p2*: P2 END ;
    R2* = RECORD p1*: P1 END ;                (*cyclic reference through record fields*)
    R3* = RECORD (R1) p3*: P3 END ;           (*cyclic reference through type extensions*)
END M.
```

Consider the case where the types defined in module *M* are re-imported by a client via an intermediate module *M0*. In this situation, procedure *ORB.InType* is recursively called for the *re-imported* types *P1, R1, P2, R2* and *R1* (in this order) and the *last* call to *ORB.InType(R, thismod, t.base)*, made while reading the type *R2*, sets *t.base* to *R1* via its variable parameter *T*. If the module anchor of the declaring module *M* were stored in the symbol file of the re-exporting module *M0 after* the type description of *R1* (as in Project Oberon 2013), the code to read this module anchor would also be executed *after* the recursive calls to *ORB.InType* for the re-imported types *P1, R1, P2, R2* and *R1:*

```
PROCEDURE InType(VAR R: Files.Rider; thismod: Object; VAR T: Type);
  ...
BEGIN Read(R, ref);
  IF ref < 0 THEN T := typtab[-ref]
  ELSE NEW(t); T := t;
    ...
    InType(R, thismod, t.base);               (*recursive call to InType, changes t.base*)
    ...
    (*code to read the module anchor*)
    Files.ReadString(R, modname);
    IF modname[0] # 0X THEN (*re-import*)
      ...
      T := obj.type;                          (*changes t.base one level up in the recursion*)
      ...
   END
  END
END InType;
```

But the code handling re-imports may *change* the field *t.base* at the preceding level of recursion, via the variable parameter *T*, to an entry in the object list of the declaring module *M*. While this does not pose an issue – as the code will eventually discard the subtree rooted at *t* anyway, if needed – our preference is to establish the data structure for re-imported module and type objects in the compiler's symbol table *before* entering any recursion. This ensures that the data structure for re-imported modules is established in the compiler's symbol table *before* any of their types are inserted into their respective object lists, thereby removing any doubt about the correctness of the algorithm, particularly when circular references among re-imported modules are present In our implementation, we have therefore decided to move the code that reads and writes the module anchor to the *beginning* of *ORB.InType* and *ORB.OutType*, as shown below:

```
PROCEDURE InType(VAR R: Files.Rider; thismod: Object; VAR T: Type);
  ...
BEGIN Read(R, ref);
  IF ref < 0 THEN T := typtab[-ref]   (*already read*)
  ELSE NEW(t); T := t; t.mno := thismod.lev; t.typmod := thismod; t.orgref := ref;
    IF ref > 0 THEN
      Files.ReadString(R, modname);   (*code to read the module anchor*)
      IF modname[0] #  0X THEN (*re-import*)
        Files.ReadInt(R, key); Files.ReadString(R, name); Read(R, orgref);
        mod := ThisModule(modname, modname, FALSE, key);
        obj := mod.dsc;   (*search type*)
```

```
        WHILE (obj # NIL) & (obj.name # name) DO obj := obj.next END ;
        IF obj # NIL THEN T := obj.type  (*type object found in object list of mod*)
        ELSE (*insert new type object in object list of mod*)
          NEW(obj); obj.name := name; obj.class := Typ; obj.next := mod.dsc;
          obj.type := t; t.mno := mod.lev; t.typobj := obj;
          t.typmod := mod; t.orgref := orgref
        END
      ELSIF typtab[ref] # NIL THEN T := typtab[ref]  (*already re-imported*)
      END ;
      typtab[ref] := T
    END ;
    Read(R, form); t.form := form;
    IF form = Pointer THEN InType(R, thismod, t.base); ...
    ELSIF form = Array THEN InType(R, thismod, t.base); ...
    ELSIF form = Record THEN InType(R, thismod, t.base); ...
    ELSIF ...
    END
  END
END InType;

PROCEDURE OutType(VAR R: Files.Rider; t: Type);
  ...
BEGIN
  IF t.ref > 0 THEN (*type was already output*) Write(R, -t.ref)
  ELSE obj := t.typobj;
    IF (obj # NIL) OR (t.form IN {Pointer, Array, Record}) THEN
      Write(R, Ref); t.ref := Ref; INC(Ref);
      IF (t.mno > 0) & (obj # NIL) THEN (*re-export*)
        Files.WriteString(R, t.typmod(Module).orgname); Files.WriteInt(R, t.typmod.val);
        Files.WriteString(R, obj.name); Write(R, t.orgref)
      ELSE Write(R, 0)
      END
    END ;
    Write(R, t.form);
    IF t.form = Pointer THEN ...
    ELSIF t.form = Array THEN ...
    ELSIF t.form = Record THEN ...
    ELSIF ...
    END
  END
END OutType;
```

11. Additional improvements

Project Oberon 2013 allows a maximum of 15 modules to be imported by any single module. This typically does not pose any issues, as it aligns with the good programming practice of structuring the module hierarchy in a way that only a small number of modules are imported. However, this upper limit also includes modules from which types are (only indirectly) *re-imported*. These modules do not necessarily have to be explicitly listed in the import statement; their imports can remain hidden. Therefore, in deep module hierarchies, one may want to lift this restriction. To address this need, our implementation increases the maximum number of modules that can be directly or indirectly imported by a single module from 15 to 63, providing greater flexibility in managing complex module structures.

To avoid searching for the module that originally exported a type in *ORB.OutType*, our implementation provides type nodes (type *TypeDesc*) with a back-pointer *typmod* to the module from which the type was originally imported. This back-pointer is used only for imported or re-imported named types. This improvement is shown in *italics* in the above code excerpts from *ORB.InType* and *ORB.OutType*.

Beyond that, we have refrained from introducing any additional "optimizations" in order to keep the code simple and understandable. An experiment has shown, for example, that various techniques for further shortening the symbol files, as described in [11], yield only marginal gains at the cost of increased implementation complexity. Compared with earlier implementations of the Oberon compiler, the symbol file produced by the Project Oberon 2013 compiler is already quite compact, leaving relatively little room for further improvement. Moreover, on modern computers, optimizing file access is no longer a high priority.

References

1. Parnas D.L. *On the Criteria To Be Used in Decomposing Systems into Modules*. Comm ACM 15, 12 (December 1972)
2. Wirth N. *MODULA-2*. Computersysteme ETH Zürich, Technical Report No. 36 (March 1980) (ch. 15 describes the use of an implementation of Modula-2 on a DEC PDP-11 computer)
3. Geissmann L. *Separate Compilation in Modula-2 and the Structure of the Modula-2 Compiler on the Personal Computer Lilith*, ETH Zürich Dissertation No. 7286 (1983)
4. Wirth N. *A Fast and Compact Compiler for Modula–2*. Computersysteme ETH Zürich, Technical Report No. 64 (July 1985)
5. Gutknecht J. *Compilation of Data Structures: A New Approach to Efficient Modula–2 Symbol Files*. Computersysteme ETH Zürich, Technical Report No. 64 (July 1985)
6. Rechenberg, Mössenböck. *An Algorithm for the Linear Storage of Dynamic Data Structures*. Internal Paper, University of Linz (1986)
7. Gutknecht J. *Variations on the Role of Module Interfaces*. Structured Programming 10, 1, 40-46 (1989)
8. J. Templ. *Sparc–Oberon. User's Guide and Implementation*. Computersysteme ETH Zürich, Technical Report No. 133 (June 1990).
9. Griesemer R. *On the Linearization of Graphs and Writing Symbol Files.* Computersysteme ETH Zürich, Technical Report No. 156a (1991)
10. Pfister, Heeb, Templ. *Oberon Technical Notes.* Computersysteme ETH Zürich, Technical Report No. 156b (1991)
11. Franz M. *The Case for Universal Symbol Files.* Structured Programming 14: 136-147 (1993)
12. Crelier R. *Separate Compilation and Module Extension*. ETH Zürich Dissertation No. 10650 (1994).
13. Wirth N. *An Oberon Compiler for the ARM Processor*. Technical note (December 2007, April 2008), www.inf.ethz.ch/personal/wirth
14. Wirth N., Gutknecht J. *Project Oberon 2013 Edition*, www.inf.ethz.ch/personal/wirth